Tiny Telemetry Protocol

Efficient Periodic Telemetry over UDP with Optional Batching

CSE361: Computer Networking

Submitted to: Prof. Ayman Bahaa & Dr. Kareem Emara

Made with GAMMA

Project Contributors

A collaborative effort by the i-Credit Hours Engineering Programs team at Ain Shams University.

Abdelrahman Adel

ID: 23P0144

Monzer Mahmoud

ID: 23P0122

Omar Ahmed Abouraia

ID: 23P0100

Hossam Osama

ID: 23P0010

Zeina Reda

ID: 23P0181

Logine Mohamed

ID: 23P0187

Made with GAMMA

Introduction to TinyTelemetry

TinyTelemetry is a lightweight, UDP-based protocol designed for periodic sensor reporting from embedded devices to a central collector.

- Prioritizes minimal overhead and predictable timing.
- Robust under lossy conditions (5–15% expected loss).
- Stateless communication with optional batching.
- Sequence-based detection of gaps and duplicates.

Made with GAMMA

TinyTelemetry vs. MQTT-SN

Why a custom protocol? TinyTelemetry is purpose-built for deterministic periodic telemetry.

Stateless Design

No sessions, ACKs, or broker registration required.

Deterministic

Avoids retransmission jitter found in MQTT-SN QoS.

Fixed Header

Uses a consistent 12-byte header for simplicity.

Made with GAMMA

Message Format & Architecture

Unidirectional Client-Server Model

START

INIT

REPORTING

END

Header Structure (12 Bytes)

- Version & MsgType (1 Byte)
- DeviceID (16-bit)
- SeqNum (32-bit)
- Timestamp (32-bit)
- Flags (8-bit)

Payload Options

Single: [Type][Float][Checksum]

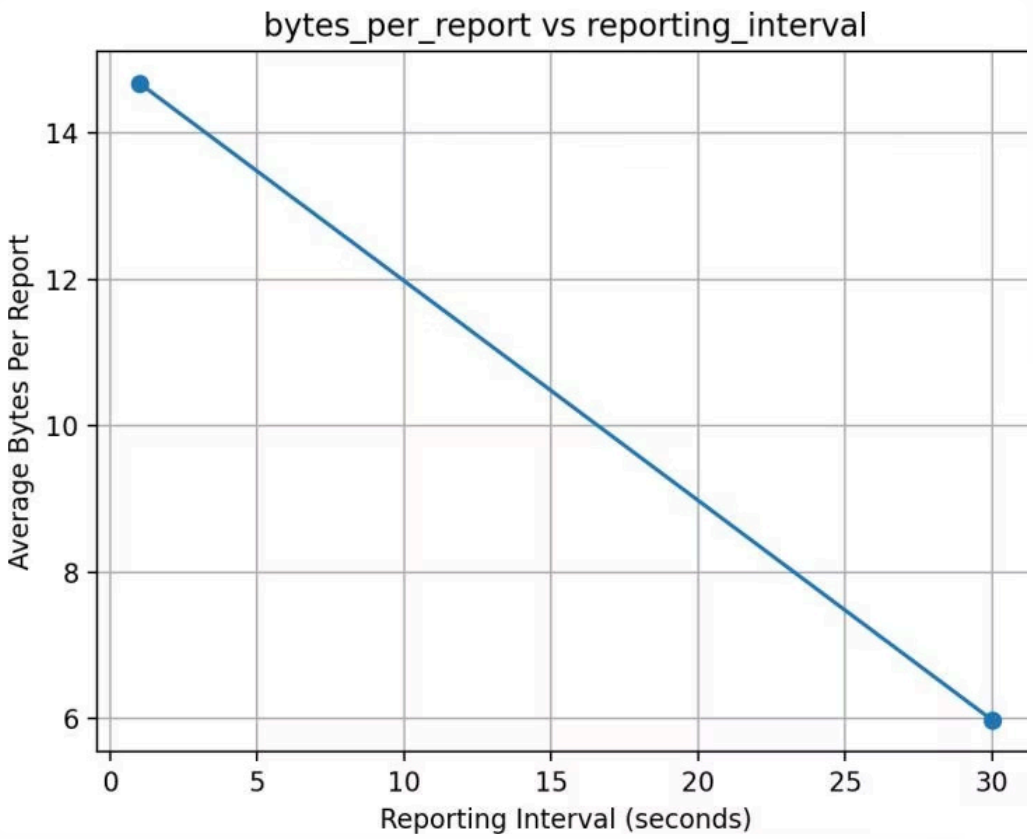
Batch: [Count] + N × [Reading]

Max packet size is constrained to 200 bytes.

Made with GAMMA

Experimental Evaluation: Efficiency

Analysis of how reporting intervals affect data overhead through batching.



Key Findings

- Average bytes per report decrease as interval increases.
- 12-byte header overhead is amortized across batched readings.
- Efficiency scales effectively for low-frequency reporting.

Made with GAMMA

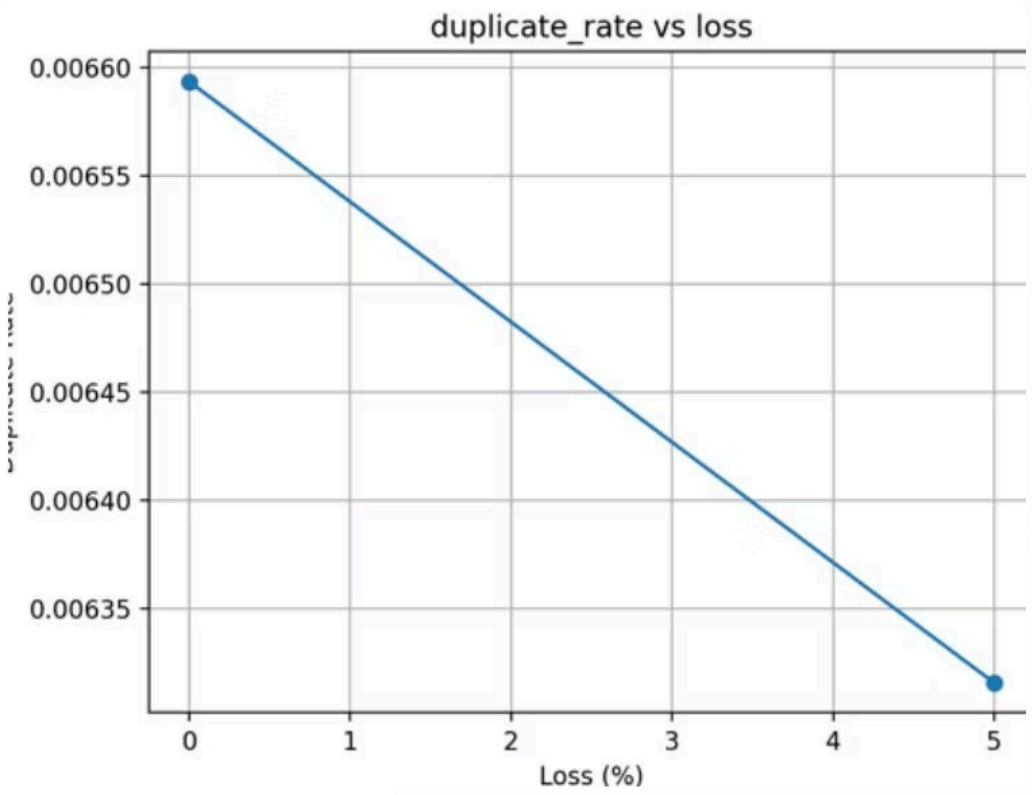
Experimental Evaluation: Packet Loss

Testing protocol behavior under injected network loss using Linux netem.

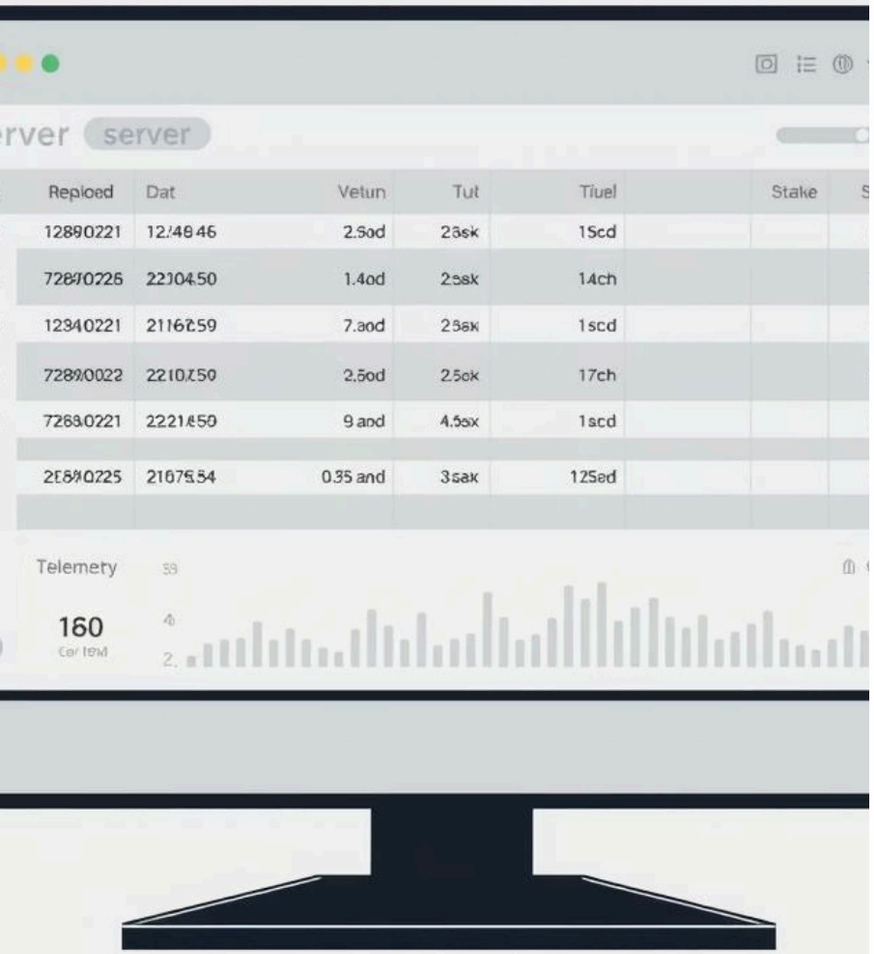
Duplicate Rate Analysis

The duplicate rate decreases as loss increases because lost packets are never retransmitted.

- 0% Loss: ~0.00660 dups
- 5% Loss: ~0.00635 dups



Made with GAMMA



Example Use Case Walkthrough

Scenario: Device 999 Temperature Reporting

01

Initialization

Client sends INIT (seq=0). Server resets tracking for Device 999.

02

Data Transmission

Client sends DATA (seq=1-10) with values like 22.3°C every second.

03

Heartbeat

After 10 packets, a HEARTBEAT is sent to signal the device is active.

04

Shutdown Summary

Server prints stats: received=60, lost=0, dups=0, resets=1.

Made with GAMMA

Reliability Features & Analytics

While TinyTelemetry avoids retransmissions, it employs smart detection and estimation.

Sequence-Based Detection

Identifies duplicates, gaps, and reordered packets without overhead.

Moving-Average Estimation

Collector approximates missing values using the mean of the last K readings for analytics.

Optional Checksum

A 1-byte modulo checksum provides basic data integrity for sensor readings.

Made with GAMMA

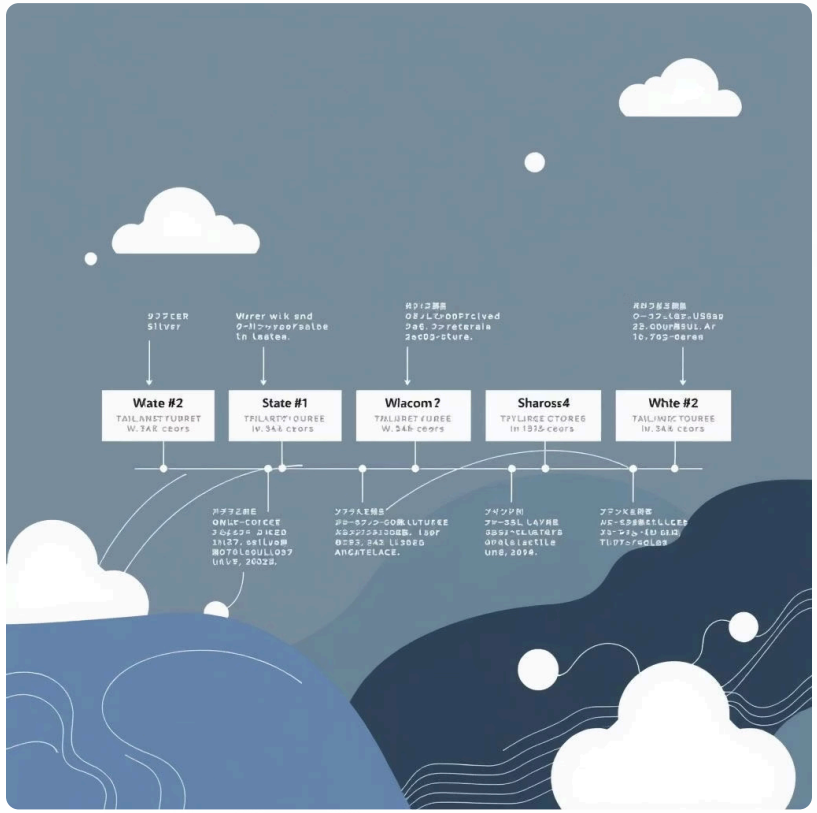
Limitations & Future Work

Current Constraints

- No end-to-end reliability or guaranteed delivery.
- Unsuitable for high-security environments (no encryption).
- Batching increases impact of a single lost packet.

Future Directions

- Lightweight selective acknowledgments.
- Message Authentication Codes (MAC) for security.
- Validation on physical embedded hardware.



References: RFC 768 (UDP), MQTT-SN (OASIS), RFC 3550 (RTP), Linux netem, Python socket/struct docs.

Made with GAMMA